B+tree implementation assignment: Internal Structure

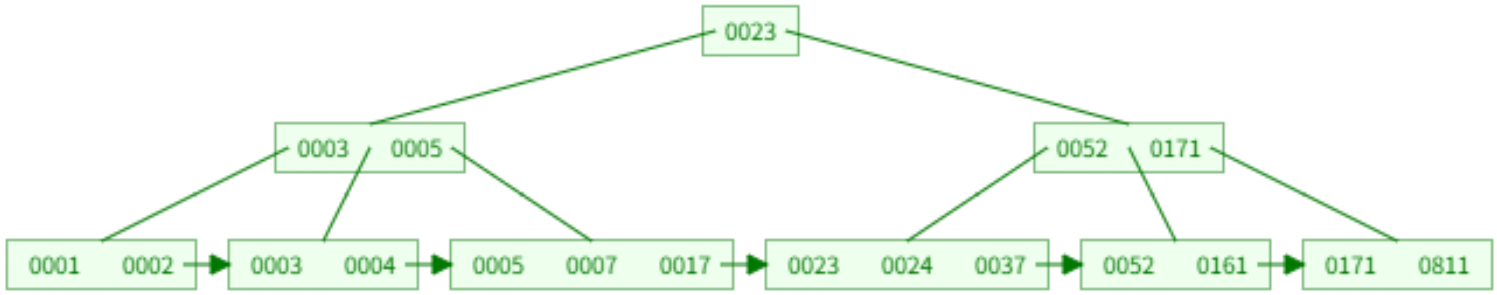
September 10, 2026

Seongeun Ryu, Jeong-Min Park, Hanseok Oh

Big-Data Science Lab.
Department of Computer Science and Engineering
Hanyang University

Constraints - Internal Structure

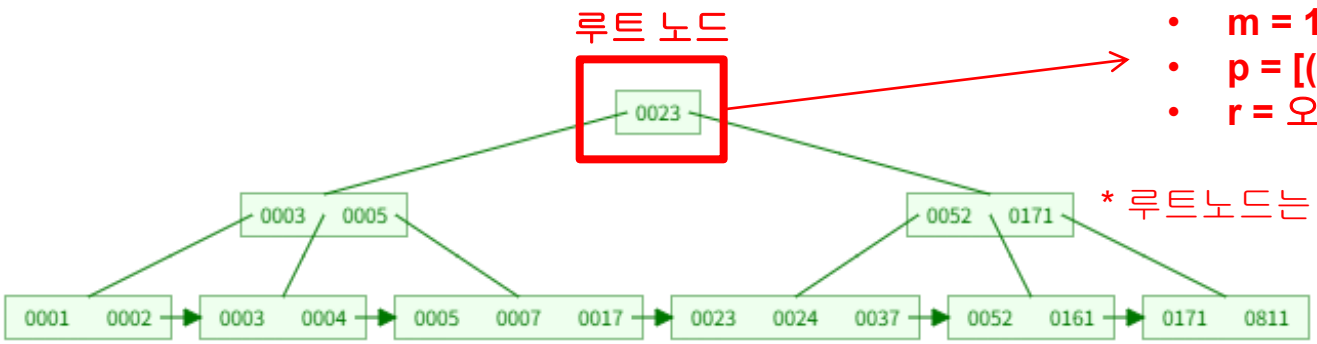
- An example of B+tree



4. Constraints - Internal Structure

- Each node of a B+ tree index should contain the following data inside:
  - Non-leaf node
    - ✔ *m*: # of keys
    - ✔ *p*: an array of *<key, left_child_node>* pairs
    - ✔ *r*: a pointer to the rightmost child node
  - Leaf node
    - ✔ *m*: # of keys
    - ✔ *p_*: an array of *<key, value(or pointer to the value)>* pairs
    - ✔ *r*: a pointer to the right sibling node

Constraints - Internal Structure



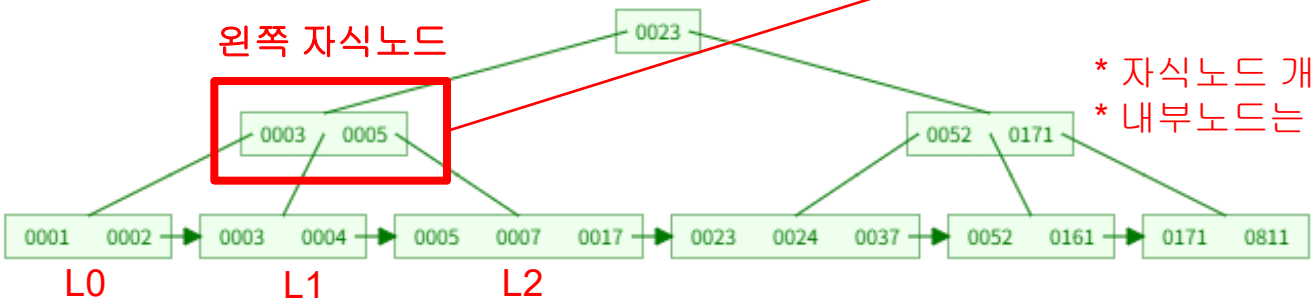
- **m = 1 (키가 하나)**
- **p = [(0023, 왼쪽자식노드 주소)]**
- **r =** 오른쪽자식노드 주소

* 루트노드는 실제 데이터(key, value) 저장 X

4. Constraints - Internal Structure

- Each node of a B+ tree index should contain the following data inside:
  - Non-leaf node
    - ✔ *m*: # of keys
    - ✔ *p*: an array of *<key, left_child_node>* pairs
    - ✔ *r*: a pointer to the rightmost child node
  - Leaf node
    - ✔ *m*: # of keys
    - ✔ *p_*: an array of *<key, value(or pointer to the value)>* pairs
    - ✔ *r*: a pointer to the right sibling node

Constraints - Internal Structure



- **m = 2**
- **p = [(0003, L0 주소), (0005, L1 주소)]**
- **r = L2 주소**

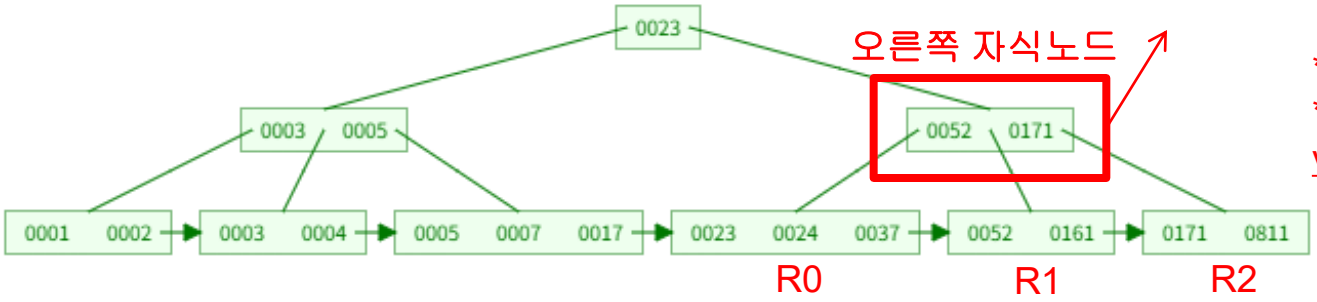
* 자식노드 개수는 m+1 = 3개
* 내부노드는 실제 데이터(key, value) 저장 X

4. Constraints - Internal Structure

- Each node of a B+ tree index should contain the following data inside:
  - Non-leaf node
    - ✔ *m*: # of keys
    - ✔ *p*: an array of *<key, left_child_node>* pairs
    - ✔ *r*: a pointer to the rightmost child node
  - Leaf node
    - ✔ *m*: # of keys
    - ✔ *p_*: an array of *<key, value(or pointer to the value)>* pairs
    - ✔ *r*: a pointer to the right sibling node

Constraints - Internal Structure

- **m = 2**
- **p = [(0052, R0 주소), (0171, R1 주소)]**
- **r = R2 주소**

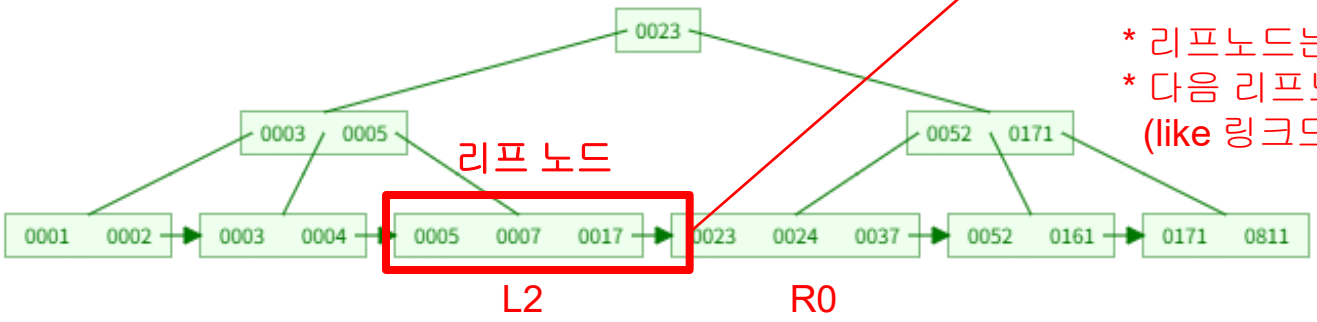


* 자식노드 개수는 m+1 = 3개
* 내부노드는 실제 데이터(key, value) 저장 X

4. Constraints - Internal Structure

- Each node of a B+ tree index should contain the following data inside:
  - Non-leaf node
    - ✔ *m*: # of keys
    - ✔ *p*: an array of *<key, left_child_node>* pairs
    - ✔ *r*: a pointer to the rightmost child node
  - Leaf node
    - ✔ *m*: # of keys
    - ✔ *p_*: an array of *<key, value(or pointer to the value)>* pairs
    - ✔ *r*: a pointer to the right sibling node

Constraints - Internal Structure



- **m = 3**
- **p_ = [(0005, v5), (0007, v7), (0017, v17)]**
- **r = 다음 리프노드(R0) 주소**

* 리프노드는 실제 데이터(key, value) 저장
* 다음 리프노드를 포인터(r)로 연결
(like 링크드 리스트)

4. Constraints - Internal Structure

- Each node of a B+ tree index should contain the following data inside:
  - Non-leaf node
    - ✔ *m*: # of keys
    - ✔ *p*: an array of *<key, left_child_node>* pairs
    - ✔ *r*: a pointer to the rightmost child node
  - Leaf node
    - ✔ *m*: # of keys
    - ✔ *p_*: an array of *<key, value(or pointer to the value)>* pairs
    - ✔ *r*: a pointer to the right sibling node

B+트리 삭제 시 리프 최소 개수 조건 및 Visualizer 결과 관련 문의

Updated 12 months ago by

A B+-tree is a rooted tree satisfying the following properties:

- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
  - If the root is not a leaf, it has at least 2 children.
  - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.

강의 자료에 따르면 leaf node에서 ⌈(n-1)/2⌉ ~ n-1 의 key, value쌍이 존재해야 한다고 나와있습니다.
만약 n이 4라면 2~3개의 key, value쌍이 나와야한다고 이해하고 있습니다.



B+ Tree visualizer에서는 위와같은 트리에서 81의 값을 delete하게 된다면 아래와 같이 1개의 key, value 쌍이 생기게 됩니다.



visualizer가 잘못된 것 같은데, 확실히 여쭤보고 싶어서 질문 남깁니다.
감사합니다!

https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html

b+tree

Instructors' Answer

Updated 12 months ago by Seongeun Ryu

안녕하세요.

말씀하신 것처럼 Visualizer 결과는 B+tree의 단순화된 시각화 참고용이며, 강의 자료 알고리즘이 엄밀하게 구현된 것은 아닙니다.
따라서, 실제 구현 과제에서는 강의 자료 규칙대로 underflow가 발생하면 병합이나 재분배를 수행하여 조건을 만족시켜야 합니다.

좋은 질문 감사합니다.
데이터베이스 팀 드림

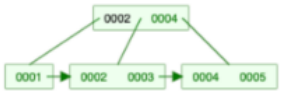
Edit 0

insert시 Split 기준 질문드립니다.

Updated 12 months ago by



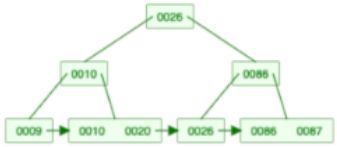
[0_Specific B+Tree]에서는 처음 ceil(n/2)노드를 현재 노드에 저장해두고, 새 노드에 그 나머지를 저장하도록 명시되어있습니다.
위 사진의 경우 리프 노드가 12/3/45로 나뉘고 부모노드는 34가 될 것 같은데,



참고한 시뮬레이터에서는 분할시 ceil(n/2)-1 개를 현재 노드로 저장하여 위와 같은 그림이 생성되었습니다.
[0_B+tree assignment internal structure] 에는 B+tree 시뮬레이터와 유사한 그림이 첨부되어 있습니다.

올려주신 내용을 바탕으로 하면 최대 키 2개일 때
26 86 87
10 20/ 26 / 86/ 87
는 87을 루트로 비대칭적인 트리가 형성되는 듯 합니다. => 혹시 이게 아닐까요?..

시뮬레이터를 참고하면



이런 식으로 동작합니다.

최대 key가 2개일 때 이를 2개/1개로 나누는 방식과, 1개/2개로 나누는 방식 모두 인정이 되는지,

제공해주신 피피티까지가 과제 명세의 일부라 해당 내용을 따라야하는지, 아니면 트리 자체의 구조 및 출력은 자율적인지 여쭤보고자 합니다.

Instructors' Answer

Updated 12 months ago by Seongeun Ryu

안녕하세요.

교재(0_Specific B+tree.pdf)에서는 split 시 앞쪽 [n/2]개를 남기는 것을 원칙으로 설명하고 있습니다.
따라서 균등하게 나누는 방식(예: 최대 키 2개일때 1/1 분할)이 정석적인 규칙입니다.

다만 시뮬레이터 구현에서는 [n/2]−1 기준을 적용하여 2/1이나 1/2로 비대칭 분할이 되는 경우도 있습니다.
이 외에도, 시뮬레이터 상에서의 삽입/삭제 에서는 교재의 제약조건과 다르게 동작하는 부분들이 있는데 실제 구현 과정에서는
과제 명세와 교재에서 설명하는 제약조건을 기준으로 구현해주시면 문제되시는 일은 없으실 것 같습니다. (시뮬레이터는 전반적인 B+tree의 개념적인 이해를 위해 제공드렸던 이유가 큽니다)

제공드린 교재에서 구체적으로 설명하지 않는 세부 구현의 경우에는 제약조건을 만족하는 선에서 어떤 방식으로 구현하셔도 무방합니다.

감사합니다.

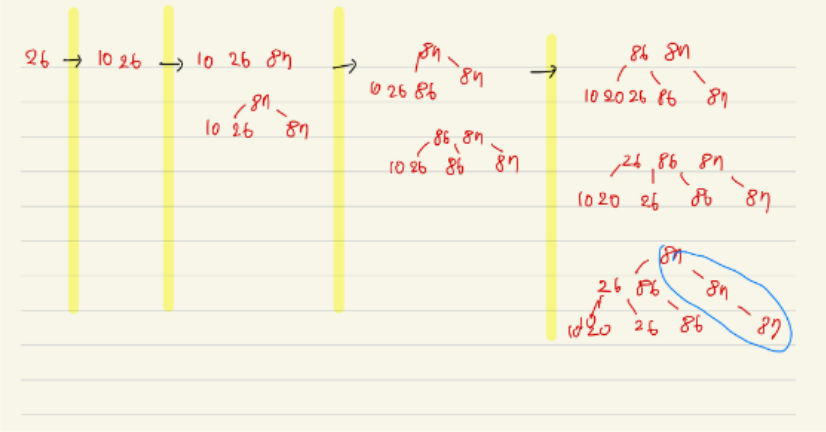
insert시 split 기준 질문 드립니다.

Updated 12 months ago by

실습 강의 자료에서 저는 [n/2] 개수만큼의 키를 기존 노드, 남은 키들은 오른쪽의 새로운 노드에 두고 새로운 노드의 가장 작은 값이 부모로 올라가는 구조라고 이해했습니다.

그럼 차수가 3일 때 2/1개로 나누어야 하는데 input.csv에 있는 숫자들을 순서대로 insert하면 그림처럼 20을 insert하고 어떻게 트리 구조를 만들어야하는지 모르겠습니다. 2/1개로 나누면 오른쪽 노드로 이동하는 키값이 항상 부모로 올라가는 형태로 이해했지만 87을 루트로 올리게 되면 비쁠트리의 구조가 깨지게됩니다.

혹시 leaf, nonleaf 노드를 나누는 방식이나 부모로 올라가는 키값을 정하는 기준이 다른 것일까요?
1/2로 나누는 방식으로 구현하면 감점이 있을지도 궁금합니다.



b+tree

Instructors' Answer

Updated 12 months ago by Seongeun Ryu

안녕하세요.

우선, non-leaf 노드와 leaf 노드의 분할 방식에는 차이가 있습니다.
leaf 노드에서는 split 시 앞쪽 [n/2]개는 기존 노드, 나머지는 새 노드에 두고, 새 노드의 가장 작은 키를 부모로 올린다는 것입니다.
그러나 non-leaf 노드에서는 split 시 B-tree와 동일하게 가운데(중간) 키를 부모로 올려 좌/우의 균형을 맞춰줍니다.
따라서, 보내주신 예시에서는 최종 구조가 다음과 같습니다:



[10,20][26][86][87]
1/2로 균등 분할하는 방식으로 구현하셔도 상관 없으십니다.

감사합니다.

Instructions for compiling: Executable file

Updated 12 months ago by (Anon. Gear to classmates)

코드와 함께 제출하는 document (PDF)의 compiling instructions를 작성하는 부분은 혹시 Windows기준으로 Compile 될 수 있도록 설명해야되나요? 또한 맥에서 컴파일된 형식을 제출하는 것은 불가할까요?
감사합니다.

b+tree

Instructors' Answer

Updated 12 months ago by Seongeun Ryu

안녕하세요,

Windows 기준으로 compile될 수 있도록 설명해주시면 감사할 것 같습니다.
(채점을 위한 PC가 Windows인 점 양해 부탁드립니다.)

감사합니다.